
PoCGYM: Evaluating AI Coding Agents for PoC Generation via Verifiable Bug Injection

Haomin Qi¹ Xiangzhe Xu² Yiming Huang¹ Jingbo Shang¹ Chengpeng Wang^{2,3}

¹University of California, San Diego ²Purdue University ³National University of Singapore

{h5qi, yih112, jshang}@ucsd.edu

{xzx, wang6590}@purdue.edu

Abstract

Proof-of-concept (PoC) generation is a central capability for AI code auditing. A practical cyber-reasoning system should not only flag that a bug or vulnerability may exist, but also produce executable evidence showing that the defect is reachable, triggerable, and observable. However, existing benchmarks either reuse public historical bugs and PoCs, raising data contamination risks and often skewing evaluation toward known memory-safety crashes, or rely on static labels and manual construction that do not scale to diverse, verifiable triggers. We present POCGYM, a fully automated framework for constructing PoC-generation benchmarks via verifiable bug injection. Specifically, POCGYM implants diverse bug types into test-reachable execution paths of real-world Java projects, validates each injected case by rebuilding the project and rerunning the associated test, and retains only cases whose injected bug is concretely triggerable. To further mitigate data contamination, POCGYM applies bug-preserving transformations that alter the surrounding structure of the injected defect while preserving its bug-triggering behavior. Leveraging 10 bug types and 6 real-world Java projects with test suites, POCGYM automatically constructs 1,300 benchmark cases. We evaluate four state-of-the-art coding agent frameworks instantiated in six framework/model pairings and find that PoC generation remains challenging across all evaluated settings. Our findings suggest that the bottleneck for AI-driven code analysis is not merely recognizing suspicious code, but reasoning over repository context and operationalizing bug hypotheses into executable PoCs for diverse bug types. Our POCGYM, the benchmark cases, and evaluation scripts are publicly available.

1 Introduction

Large language model (LLM) agents [1, 2, 3] and agentic coding systems [4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14] are increasingly being applied to high-stakes software engineering tasks [15], making reliable code auditing an important capability for such systems [16, 17]. In real-world AI coding scenarios, the key question is not only whether an agent can point to suspicious code, but also whether it can provide evidence strong enough for developers to trust and act on [18, 19, 20, 21, 22, 23, 24]. This requirement matters because bug reports generated by existing AI coding agents may still contain non-negligible false positives [25, 26], and repeated false alarms can quickly erode developers’ trust in the tool. For this reason, a useful coding agent should do more than report that a bug may exist. It should generate an executable proof of concept (PoC), such as a test case or concrete input, that triggers the bug and makes its effect observable [18, 27]. Such a PoC turns a speculative warning into actionable evidence. It helps developers distinguish real defects from false alarms and increases their confidence that the report deserves attention.

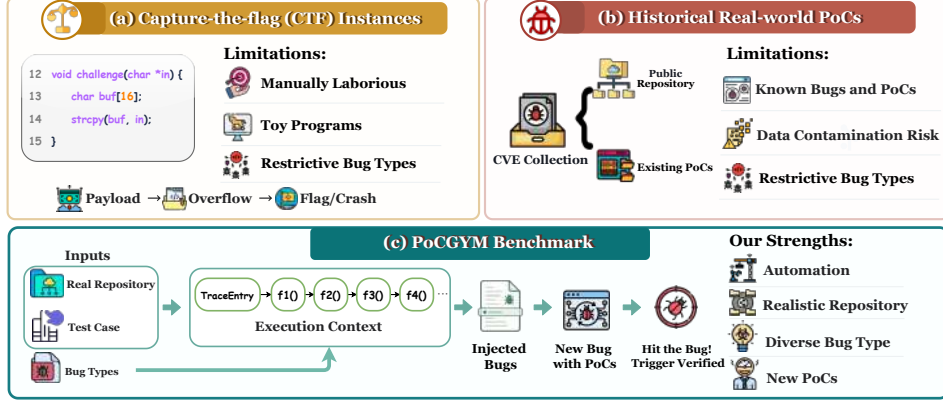


Figure 1: Comparison of benchmarking methodologies for PoC generation.

Unfortunately, existing benchmarks remain inadequate for evaluating the PoC-generation capabilities of AI coding agents. As shown in Figure 1(a) and (b), these benchmarks generally fall into two broad categories. First, CTF benchmarks such as NYU CTF Bench [28] and Cybench [29] offer executable targets and clear success criteria but rely on self-contained settings that fail to capture the complexity of real software repositories. Moreover, their construction typically involves significant manual effort to curate and containerize specific challenges, making them difficult to scale or refresh. Second, benchmarks built from real-world CVEs [20], bug-bounty reports [19], or fuzzing artifacts [18] improve realism yet typically reuse public bugs and PoCs, introducing data-contamination risks. The latter also tend to be narrow in bug semantics. CyberGym [18], for example, focuses on C/C++ memory-safety bugs, leaving logic errors and state-dependent failures underrepresented. Neither category jointly provides automatic construction, realistic repository context, diverse bug semantics, and reduced data contamination.

These limitations highlight the need to move beyond static collections of existing bug cases and public PoCs, and instead construct benchmarks dynamically and automatically as models evolve [30, 31, 32]. To make such benchmarks meaningful for realistic bug detection, they should capture the key challenge faced by AI coding agents in real-world code auditing: reasoning over repository context. Prior studies and practical code-auditing workflows suggest that even when the defective logic follows a relatively simple pattern, successful detection and PoC generation often depend on whether the agent can understand the surrounding execution context, including data dependencies, object states, call chains, and control-flow paths that make the bug reachable and observable.

This insight motivates our benchmark-construction strategy. We synthesize benchmark cases by implanting representative bug semantics into real program contexts that are already exercised by existing tests. This design separates the bug type from the contextual reasoning required to expose it: even when the injected defect follows a simple bug type, generating a valid PoC requires understanding the data dependencies, object states, call chains, and control-flow paths that make the bug reachable and observable.

Specifically, we introduce POCGYM, a fully automated framework for constructing PoC-generation benchmarks via verifiable bug injection. POCGYM starts from two ingredients, namely a bug type that specifies faulty semantics, and a real execution context collected from a test in a real-world Java project. We derive bug types from CodeQL Java query libraries [33], adapting their bug semantics into concrete bug-injection specifications that cover API Contract, Value Flow, and Logic failures. Taking such a bug type as input, POCGYM injects buggy code into a test-reachable production-code path in the real-world project, rebuilds the project, reruns the associated test, and retains only cases in which the injected bug is concretely triggered. To further mitigate data contamination, POCGYM applies bug-preserving transformations after injection. These transformations alter the surrounding structure of the injected defect by modifying data flow, control flow, and function call structure, while preserving the program behavior needed for the reference PoC to trigger the bug. Ultimately, the resulting benchmark cases are grounded in real repository context, paired with executable reference PoCs.

We implement POCGYM and automatically construct 1,300 executable benchmark cases using ten bug types and six real-world Java projects with test suites, at a total cost of \$16,500. We eval-

uate four state-of-the-art agent frameworks instantiated in six framework/model pairings on their ability to generate PoCs for the constructed benchmark cases. Our evaluation shows that the performance of current coding agents remains far from satisfactory on this task. Even the best-performing agent, Codex with GPT-5.4, achieves an average success rate of only 68.9%. Performance varies substantially across agents and models, with the largest gap in average success rate reaching 50%. Moreover, all evaluated coding agents consistently struggle more with Value Flow and Logic bugs than with API Contract bugs. Finally, we observe that longer execution contexts and greater transformation depth consistently increase the difficulty of PoC generation and reduce the success rates of different agents. The overall evaluation costs \$8,700 in total.

Overall, this work makes three main contributions. First, we present POCGYM, a systematic methodology for automatically constructing PoC-generation benchmarks by injecting diverse types of bugs into real-world projects, where the generated bugs and their corresponding PoCs do not already exist in public repositories. Second, we instantiate this methodology using diverse bug types derived from the CodeQL Java libraries and real-world Java projects, yielding a large-scale PoC-generation benchmark with reference PoCs. Third, we conduct an extensive evaluation of state-of-the-art coding agents on the resulting benchmark, producing insights that can guide future research on agentic PoC generation.

2 Background and Motivation

PoC Generation. Proof-of-concept (PoC) generation is the task of producing an executable artifact that demonstrates a concrete software defect [23, 24, 34]. Depending on the program and bug type, a PoC may take the form of a failing unit test, a crafted input, a request sequence, or another runnable witness [25, 35, 36]. Unlike a natural-language bug report [17], which only states that a defect may exist, a PoC shows that the defect can be reached under concrete execution, triggered by a specific setup, and observed through a failure signal such as an exception, assertion violation, or crash [25].

This capability is central to practical code auditing [25, 23, 37, 38, 18, 39, 35]. In real-world workflows [40, 41], the cost of a reported bug is not only in noticing suspicious code, but in determining whether the finding is real and actionable. Developers and security analysts must often determine whether the reported path can actually be reached and what input or interaction sequence trigger the defect. A PoC answers these questions by turning a bug hypothesis into executable evidence.

PoC generation is also a substantially harder task than static bug detection [18, 38]. To generate a valid PoC, an agent must reason about the surrounding program context, including the call stack and the path conditions required to reach the bug, and instantiate concrete program values that drive execution toward the defective path. Small mistakes in these values may steer execution away from the path that exposes the bug. Hence, PoC generation provides a stronger and more operational measure of auditing capability than simply asking whether an agent can identify suspicious code.

Limitations of Existing Benchmarks. As shown in Table 1, existing PoC-generation benchmarks support executable evaluation but do not jointly provide realistic project context, automated construction, diverse bug semantics, and new PoCs as evaluation targets. CTF benchmarks offer clear success criteria, but are often manually constructed around small programs and restricted bug types [28, 29, 42]. Historical-vulnerability benchmarks improve realism by using real projects and bugs [20, 38, 18], but they rely on public vulnerability artifacts, creating contamination risks and often emphasizing reproduction of known memory-safety PoCs. These limitations motivate an automated benchmark-construction technique that generates execution-validated, contamination-free PoC-generation cases with diverse bug semantics in real-world project contexts.

Our Goal. To close this gap, we define four requirements for PoC-generation benchmarks: (1) **Automation**, i.e., cases can be constructed without per-case manual bug design, labeling, or PoC authoring; (2) **Realistic project context**, i.e., the tasks should arise from real-world projects whose scale and structure reflect practical software engineering settings; (3) **Diverse bug semantics**, i.e., the evaluation should extend beyond memory-safety-only defects to include logic errors and other various behaviors; (4) **Mitigated data contamination**, i.e., the benchmark should reduce reliance on public historical vulnerabilities and their corresponding PoCs.

Table 1: Comparison of AI-agent cybersecurity benchmarks. **Scope** indicates whether the benchmark is based on capture-the-flag tasks (CTF), real-world projects (RW), or both. **Diversity** summarizes the diversity of bug types covered by the benchmarks. Specifically, high indicates coverage of multiple distinct defect families beyond a single oracle or vulnerability class. Medium indicates broader-than-single-family coverage with a dominant bug type or task format. Low indicates a narrowly scoped defect family or benchmark task. **Automation** indicates whether benchmark cases can be generated without per-case manual bug design or PoC authoring. **New PoCs** indicates whether the benchmark avoids direct reuse of public PoCs of existing bugs.

Benchmark	Scope	Diversity	Automation	New PoCs
NYU CTF Bench [28]	CTF	High	✗	✗
Cybench [29]	CTF	High	✗	✗
CyberExplorer [42]	CTF	Medium	✗	✗
AutoAdvExBench [43]	CTF+RW	Low	✗	✗
AutoPenBench [44]	CTF+RW	Medium	✗	✗
CVE-Bench [20]	RW	Medium	✗	✗
BountyBench [19]	RW	High	✗	✗
SEC-bench [38]	RW	Medium	✓	✗
CyberGym [18]	RW	Medium	✓	✗
POCGYM (Our Work)	RW	High	✓	✓

These four requirements define an evaluation setting that is scalable, realistic, semantically broad, and less vulnerable to data contamination. Meeting them is essential for assessing whether agents can generate an unseen PoC, rather than merely recognizing patterns or reproducing existing ones.

3 Methodology

This section introduces POCGYM, a framework for constructing benchmarks to evaluate AI coding agents for PoC generation. Given a real-world project with an existing test suite and a library of bug types, POCGYM synthesizes new benchmark cases by injecting bugs into an executable program contexts and validating that the injected bugs can be triggered by concrete test inputs. We refer to such a triggering test input as the *reference PoC* of the generated case.

Figure 2 illustrates the overall workflow of POCGYM. The central idea of POCGYM is to use functions exercised by existing test cases as injection contexts. Since these functions are already reached during test execution, injecting bugs into them substantially increases the likelihood that the resulting bugs are reachable and triggerable. POCGYM further performs injection and transformation in an execution-guided manner. Each generated case is validated by rebuilding the project and re-running the associated test case. In the following, we describe the three main stages of POCGYM, namely execution context collection, buggy code injection, and bug-preserving code transformation.

3.1 Execution Context Collection

Given a real-world project and its existing test suite, POCGYM first identifies the functions exercised by each test case. This step provides a test-grounded search space for bug injection. To collect the executed functions during runtime, POCGYM applies program instrumentation to the project. Specifically, it parses the source code of the project and inserts logging calls at the entry of each function. During test execution, these logging calls record the executed functions and their source locations. For a function f , we denote its source location as $\ell(f) = (\text{name}(f), \text{path}(f), \text{line}(f))$, where $\text{name}(f)$ is the function name, $\text{path}(f)$ is the path of the source file containing f , and $\text{line}(f)$ is the starting line number of f . Formally, executing a test case t yields an ordered execution log $\log \tau_t = [\ell_1, \ell_2, \dots, \ell_n]$, where each ℓ_i is the source location of a function observed in the log. We define the execution context of t as the set of unique function locations appearing in this log, $EC_t = \{\ell_i \mid \ell_i \in \tau_t\}$. Repeating this process for all test cases produces an *execution context map*, $\mathcal{M}_{EC}$, which maps each test case to its execution context $\mathcal{M}_{EC}(t) = EC_t$.

The execution context map serves as the basis for selecting candidate injection locations. Rather than injecting bugs into arbitrary files or functions, POCGYM restricts the search space to functions known to be executed by a specific test case. This design improves the likelihood that the injected bug is reachable and reduces the cost of subsequent execution-guided injection.

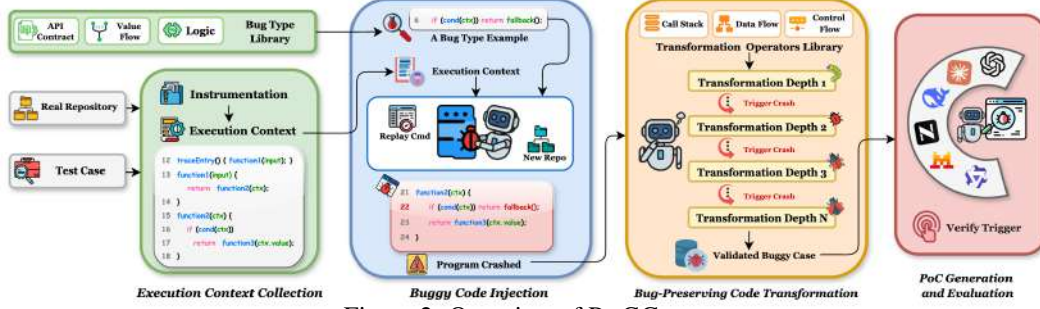


Figure 2: Overview of POCGYM.

3.2 Buggy Code Injection

In the second stage, POCGYM utilizes the collected execution contexts to synthesize benchmark cases via buggy code injection. To inject diverse types of bugs, POCGYM constructs its bug type library from CodeQL Java query examples [45]. These examples contain compact code snippets that characterize representative bug-inducing types. We organize them into three categories, namely *API Contract types*, which capture violations of API contracts or specifications; *Value Flow types*, which capture value-flow constraint violations (e.g., dereferencing null values); and *Logic types*, which capture domain-specific functional bugs. These categories span diverse manifestations of bugs in real-world programs and provide heterogeneous injection targets for benchmark construction. In our setting, accepted bugs are required to produce observable runtime failures when triggered, which provides an automatic execution oracle for validating PoCs. The bug types used in our benchmark construction are listed in Table 5 in Appendix B.

Based on the above ingredients, the buggy code injection can be formulated as follows. Given a test case t , an execution context EC_t , and a bug type b , POCGYM leverages a coding agent to inject b into the execution context EC_t of test case t . Since the functions in the execution context EC_t are exercised, the injected bug is likely to be reachable by the corresponding reference PoC, i.e., the test case t . After each injection attempt, POCGYM rebuilds the modified project and reruns the associated test case t . If the bug is not triggered, POCGYM provides the execution feedback to the coding agent and requests a refinement. This process continues until the bug is successfully triggered or the attempt budget is exhausted. This validation procedure ensures that accepted benchmark cases are not merely syntactic mutations of source code. Instead, each accepted case contains a bug that is present in the program, reachable from a concrete test case, and empirically triggerable by the reference PoC. The system prompt and user prompt used in buggy code injection are provided in Appendix G.1 and Appendix G.2.

3.3 Bug-Preserving Code Transformation

Since the bug types are derived from public CodeQL examples, directly reusing these snippets may produce benchmark cases that are syntactically or structurally similar to publicly available bug types. To alleviate this issue, POCGYM applies bug-preserving transformations that are similar in spirit to code obfuscation. Specifically, they alter the surface representation and contextual structure of the injected bug while preserving its triggering behavior under the same reference PoC. Notably, they are not required to preserve the full semantics of the original program. Instead, their goal is to reduce surface similarity to public bug examples and increase contextual complexity without changing the buggy semantics.

Technically, POCGYM is built on three families of transformation operators, which manipulate call stack, data flow, and control flow structures. The detailed definitions of the transformation operators are listed in Table 6 in Appendix B. Similar to buggy code injection, the transformation stage is also execution-guided so that we can validate whether the transformed code preserves the injected bug. If not, the transformation is reverted. Through this process, POCGYM can derive multiple benchmark variants from the same injected bug. These variants share the same underlying bug type and reference PoC, but differ in their relevant call stack, data flow, and control flow structures, enabling more diverse and challenging evaluation of AI coding agents for PoC generation. The prompt of the bug-preserving code transformation is given in Appendix G.3.

4 Evaluation

We instantiate POCGYM on six real-world Maven-based Java projects and ten observable bug types from CodeQL [45], spanning API Contract, Value Flow, and Logic defects. The resulting benchmark contains 1,300 buggy cases across four execution-context-length categories and four transformation-depth levels, and costs approximately \$16,500 to construct. Implementation details and benchmark statistics are provided in Appendix B and Appendix C.

4.1 Evaluation Setup

PoC Generation Task. We evaluate every case under two settings that correspond to two important software-engineering scenarios. In the context-agnostic setting, denoted NoEC, the agent receives the buggy repository, the natural language description of the bug type, the buggy location, but no execution context. This setting mimics the validation of static bug-detection results, where developers may have a localized bug report but no calling context showing how the buggy location is reached. In the context-guided setting, denoted WithEC, the agent receives the same inputs together with the execution context. This setting mimics the bug-reproduction and debugging workflows, where a crash stack or recorded execution trace is available as a dynamic hint. In the current benchmark instantiation, execution-context length ranges from 1 to 150 distinct production-side functions, grouped into four categories: *short* ([1, 40]), *mid-short* ([41, 70]), *mid-long* ([71, 120]), and *long* ([121, 150]). Appendix G.4 and Appendix G.5 provide the prompts used in the PoC-generation.

Agent Settings. Table 2 lists the six evaluated framework/model pairings. They cover two industrial agent frameworks (Codex and Claude Code) and two open-source agent frameworks (OpenHands and OpenCode) with representative open-weight models. The underlying models include GPT-5.4, Claude Sonnet 4.5, DeepSeek-V3.2, ZAI GLM-4.7, Devstral 2, and Qwen3-Coder-480B-A35B-Instruct. Although we do not exhaustively enumerate every framework/model combination, these six pairings provide strong industrial baselines together with diverse open-source/open-weight ones.

Table 2: The list of agents.

Agent	Framework	Model
A1	Codex	GPT-5.4
A2	Claude Code	Claude Sonnet 4.5
A3	OpenHands	DeepSeek-V3.2
A4	OpenCode	ZAI GLM-4.7
A5	OpenHands	Devstral-2-123B
A6	OpenCode	Qwen3-Coder-480B-A35B-Instruct

Each case allows at most three PoC-generation attempts. Each attempt uses a 1,800-second agent timeout, a 1,200-second replay-verification timeout, a 600-second optional formatting timeout, and a 4,200-second case-level timeout. For OpenHands, we explicitly set the temperature to 0 and the maximum output length to 4,096 tokens. For Codex, Claude Code, and OpenCode, we keep the interface-default decoding behavior rather than forcing wrapper-specific overrides for temperature, nucleus sampling (top- p), top- k , or maximum output tokens, because those controls are not exposed in a directly comparable way across all evaluated interfaces. Runs that exceed the case-level timeout, fail replay verification, or fail to materialize an executable PoC test are counted as failures. In total, the full evaluation costs approximately \$8,700. Appendix H provides the reproduction commands.

4.2 RQ1: How Well do Coding Agents Generate PoCs?

To control evaluation cost, we randomly sample 900 cases from the full 1,300-case benchmark and evaluate all six agents under both the NoEC and WithEC settings. Table 3 shows substantial performance differences: the two industrial agent frameworks, Codex (A1) and Claude Code (A2), are strongest at 68.9% and 59.4% average success, respectively, whereas the open-source settings remain substantially weaker overall. Among the open-source agents, OpenHands with DeepSeek-V3.2 (A3) outperforms OpenCode with Qwen3-Coder-480B-A35B-Instruct (A6) by 29.7 percentage points on average. End-to-end PoC generation therefore remains difficult even for the strongest agents, which still fail on more than one quarter of the benchmark cases.

Table 3: Success rates of different agents

ID	NoEC	WithEC	Average
A1	67.1%	70.7%	68.9%
A2	57.1%	61.8%	59.4%
A3	46.0%	49.8%	47.9%
A4	44.2%	47.6%	45.9%
A5	30.2%	33.8%	32.0%
A6	17.1%	19.3%	18.2%

Table 4: The numbers of successful cases in the PoC generation across different bug types. The columns L and S denote the statistics collected in the long and short execution contexts, respectively. Each cell A/B indicates the numbers of successful cases in WithEC and NoEC settings, respectively.

Bug Type	A1		A2		A3		A4		A5		A6	
	L	S	L	S	L	S	L	S	L	S	L	S
Comparison of Identical Values	18/17	22/21	17/14	19/16	14/10	15/15	11/11	15/13	9/7	10/9	4/4	6/5
Typo in equals	18/15	21/21	14/13	16/18	12/13	15/13	10/10	12/11	8/7	9/10	5/3	6/5
Typo in hashCode	16/13	18/17	16/11	19/14	10/9	15/12	11/8	13/14	7/6	10/7	4/4	5/5
Hashed Value Without hashCode	14/14	19/17	15/13	16/15	10/11	15/13	9/9	14/10	8/6	10/7	4/3	5/4
Inconsistent equals and hashCode	17/15	18/20	13/12	18/17	12/9	12/13	12/9	13/11	6/6	10/9	3/4	5/4
Intra-Function Null Dereference	19/16	23/23	14/13	19/15	9/9	12/11	11/9	13/12	9/8	10/11	7/6	8/6
Collection Element Null Dereference	16/16	20/17	14/12	16/17	10/8	12/11	8/7	11/10	8/6	11/8	5/4	8/8
Guard Predicate Inversion	16/14	18/16	13/13	17/14	13/10	17/13	12/9	14/13	7/7	8/8	4/3	5/5
Skipped Aggregation Counter	16/18	23/20	15/12	21/17	12/12	15/15	12/12	17/16	8/5	10/8	4/4	5/5
In-Bounds Collection Misrouting	15/12	17/18	11/11	13/15	12/12	14/15	12/11	14/13	6/6	8/7	4/3	5/5

Execution context consistently improves performance, but only modestly: across the six agents, WithEC outperforms NoEC by 2.2–4.7 percentage points. This suggests that execution context helps agents localize relevant functions and program paths, but bug type diversity and transformation-induced structural complexity still make the context hard to operationalize during test synthesis.

Finding 1: Current coding agents still struggle with end-to-end PoC generation, even when execution context information is available.

4.3 RQ2: How do Bug Types Affect PoC Generation?

To investigate the effect of bug types, we analyze a 500-case subset drawn from the full benchmark. For each bug type, the subset contains 25 short-context cases and 25 long-context cases, all constructed with the same depth-3 transformation configuration and the same transformation operator sequence, and each case is evaluated under both the NoEC and WithEC settings. As shown in Table 4, bug type difficulty varies substantially across the six agents. In particular, the Codex agent framework with GPT-5.4 (A1) and the Claude Code agent framework with Claude Sonnet 4.5 (A2) are relatively weak on Logic bugs, with WithEC success rates of 70.0% and 60.0%, respectively. The open-source agents exhibit a different profile. Their performance varies sharply across bug families, and they are generally less effective on Value Flow and Logic bugs than on API Contract bugs. It is worth noting that the gap between industrial and open-source agents is not uniform across all bug types. In particular, OpenHands with DeepSeek-V3.2 (A3) is consistently stronger than the other open-source agents with open-weight models evaluated here. It even narrows the gap to Claude Code with Claude Sonnet 4.5 (A2) on several logic bug types, such as Guard Predicate Inversion and In-Bounds Collection Misrouting. This suggests that open-source agent frameworks paired with strong open-weight models can approach industrial agents in specific PoC-generation scenarios.

We further quantify the effect of execution context across bug types. Figure 3 shows the difference in the number of successful cases between WithEC and NoEC for each agent–type pair. Because this comparison holds the depth-3 transformation configuration fixed across bug types, the remaining differences primarily reflect bug semantics and how effectively each agent can exploit the same context signal. The figure shows that execution context does not improve performance uniformly across agents and bug types. One plausible explanation is that, once the execution context is provided, the remaining difficulty of PoC generation lies in reasoning about the program properties within

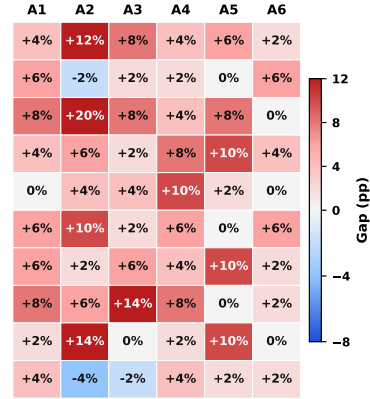


Figure 3: The difference in the numbers of successful cases under WithEC and NoEC settings for each agent–type pair. The y-axis lists the ten bug types in the same order as Table 4.

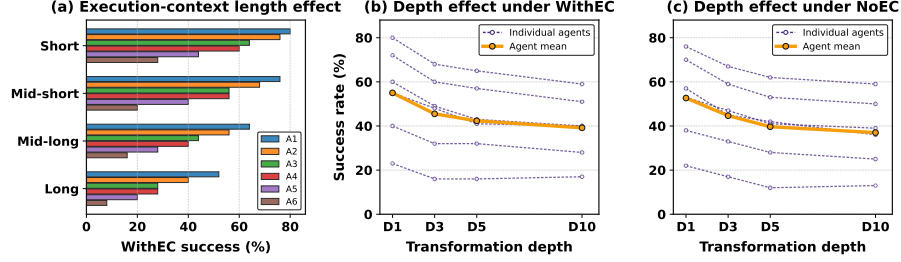


Figure 4: The effect of execution context length (a) and transformation depth (b and c).

the execution context. Because different bug types require different forms of semantic reasoning, agents and models differ in how effectively they can exploit the same contextual information. This observation has an important implication that benchmarks should cover diverse bug categories and bug types, so that they stress program properties at different granularities and levels of complexity, rather than focusing on only a narrow class of reasoning tasks.

Finding 2: Bug types affect the PoC-generation performance of the evaluated agents. Logic bugs are generally challenging for these agents. Open-source agent frameworks with open-weight models can approach industrial agents on specific bug types in PoC generation.

4.4 RQ3: How does Execution Context Length Affect PoC Generation?

To investigate the effect of execution context length, we fix the bug type to Guard Predicate Inversion and analyze a 400-case subset drawn from the full benchmark. The subset is stratified by four execution-context-length buckets and four transformation depth settings, with 25 cases in each length-by-depth cell. For each depth setting, all cases use the same transformation-operator sequence, and each case is evaluated under both the NoEC and WithEC settings.

The evaluation results show a consistent trend across both settings that longer execution contexts reduce PoC-generation success rates for all evaluated agents. For example, Figure 4a shows this trend under the WithEC setting. Considering the cases in short and long execution contexts, the success rate of Codex with GPT-5.4 (A1) decreases from 80% on short execution contexts to 52% on long execution contexts. The success rate of the Claude Code agent framework with Claude Sonnet 4.5 (A2) decreases by 36%. The open-source agent frameworks and open-weight models follow similar trends. For example, the success rate of OpenHands with Devstral-2-123B (A5) decreases from 44% to 20% in the short and long execution contexts. This suggests that execution context information can help agents narrow down relevant functions involved in the triggering process. However, such guidance does not eliminate the difficulty of constructing executable PoCs. As execution context length increases, agents must reason about more intermediate states, more setup choices, and longer dependency chains before exposing the injected bug through a valid test. Appendix D provides more detailed statistics and discussions on the effect of execution context length.

Finding 3: Longer execution contexts substantially reduce PoC-generation success rates. Even with execution context guidance, AI coding agents still struggle to generate executable PoCs as execution context length increases.

4.5 RQ4: How does Transformation Depth Affect PoC Generation?

To investigate the effect of transformation depth, we also fix the bug type to Guard Predicate Inversion and reuse the same 400-case subset, but here we regroup the cases by four transformation depth levels—1, 3, 5, and 10 applied transformations. Each depth-by-length cell again contains 25 validated cases, and the transformation operator sequence is fixed. Also, each case is evaluated under both the NoEC and WithEC settings.

As shown in Figure 4b and c, applying more bug-preserving transformations increases the difficulty of PoC generation in both settings. For example, under the WithEC setting, the success rate of Codex with GPT-5.4 (A1) decreases from 80% at depth 1 to 59% at depth 10, and the success rate of the Claude Code agent framework with Claude Sonnet 4.5 (A2) decreases from 72% to 51%. Open-

source agent frameworks with open-weight models exhibit similar trends. For instance, the success rate of OpenHands with DeepSeek-V3.2 (A3) decreases from 60% at depth 1 to 40% at depth 10.

This trend suggests that deeper transformations make the injected bug harder to expose through an executable PoC. As more transformations are applied, the bug is structurally embedded through more sophisticated code fragments, intermediate program states, and dependency relations. The agent must therefore reason through the transformed program structure before it can construct a valid test setup and oracle. The trend is not perfectly monotonic in every case, but the global downward trend indicates that transformation depth provides a practical mechanism for adjusting PoC-generation difficulty. The detailed statistics across transformation depth buckets are shown in Table 8 in Appendix E, and additional operator-level effects are discussed in Appendix F.

Finding 4: Deeper bug-preserving transformations can increase the difficulty of PoC generation, suggesting that transformation depth serves as a useful benchmark dimension for controlling PoC-generation difficulty.

5 Related Work

PoC Generation in Agentic Code Auditing. Recent coding agents can navigate repositories, execute tests, and refine hypotheses, enabling code auditing beyond earlier snippet-level classification approaches [46, 47, 16, 48]. However, their findings may suffer from incomplete context, uncertain reasoning, and hallucinated explanations, leading to false positives. PoC generation provides a concrete validation step: after detecting a potential vulnerability, an agent must produce an executable artifact that triggers it [41, 40]. Thus, PoC generation is a stronger measure of auditing capability than static bug detection alone. Recent work explores agentic PoC generation with debugging tools, execution feedback, and task-specific memory [23, 24], but our results show that current techniques still struggle with complex contexts and diverse bug types, especially logic bugs.

Benchmarks for Agentic Code Auditing. Benchmarking AI code auditing requires large-scale datasets with verified ground truth. Existing bug-detection and PoC-generation benchmarks often rely on curated vulnerability collections or public vulnerability artifacts [49, 50, 28, 38]. These sources limit bug diversity, risk data contamination, and often measure reproduction of known bugs rather than generation of fresh PoCs. Many benchmarks also emphasize C/C++ memory-safety errors because sanitizer crashes provide convenient oracles, leaving broader targets such as logic errors underrepresented [18]. POCGYM addresses these limitations through an automated pipeline that injects diverse bug types into real-world projects and validates each case by execution, improving scalability while reducing leakage from historical vulnerabilities.

6 Conclusion

This paper introduces POCGYM, a fully automated framework for constructing benchmarks for PoC generation. By injecting diverse bug types into real-world projects and retaining only concretely triggerable cases, POCGYM enables a more realistic evaluation of whether AI coding agents can turn bug hypotheses into reproducible failures. Our empirical findings reveal that existing AI coding agents share common bottlenecks when analyzing complex contexts and diverse bug types. We believe POCGYM provides a step toward a more rigorous evaluation standard and can support future research on more reliable, evidence-grounded AI code auditing systems.

7 Limitations

POCGYM has three main limitations. First, because it injects bugs into test-induced execution contexts, the generated programs may differ from naturally developed buggy code, potentially affecting agents’ reasoning and measured PoC-generation performance. Second, transformation depth offers only coarse difficulty control, as LLM nondeterminism limits the precision of the current transformation framework. Developing more controllable transformations remains future work. Third, our experiments are limited to Maven-based Java projects, which may introduce ecosystem-specific bias. Nevertheless, the methodology is general and can be extended to other languages and ecosystems with executable test environments.

References

- [1] BCG. Momentum builds, but gaps remain, 2025.
- [2] McKinsey. The state of AI, 2025.
- [3] McKinsey. Superagency in the workplace: Empowering people to unlock AI’s full potential, 2025.
- [4] OpenHands. The Open Platform for Cloud Coding Agent, 2026.
- [5] OpenCode. OpenCode | the open source AI coding agent, 2026.
- [6] Anthropic. Claude Code overview, 2026.
- [7] Augment Code. Augment Code, 2026.
- [8] Smolagents. Hugging Face Smolagents, 2026.
- [9] OpenAI. Harness engineering: leveraging Codex in an agent-first world, 2026.
- [10] Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-swe-agent: Can software engineering agents self-evolve on the fly? *arXiv preprint arXiv:2511.13646*, 2025.
- [11] Antonis Antoniadis, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. Swe-search: Enhancing software agents with monte carlo tree search and iterative refinement. *arXiv preprint arXiv:2410.20285*, 2024.
- [12] Nghi DQ Bui. Building AI coding agents for the terminal: Scaffolding, harness, context engineering, and lessons learned. *arXiv preprint arXiv:2603.05344*, 2026.
- [13] Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. Improving the efficiency of LLM agent systems through trajectory reduction. *arXiv preprint arXiv:2509.23586*, 2025.
- [14] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- [15] Anthropic. Harness design for long-running application development, 2026.
- [16] Jinyao Guo, Chengpeng Wang, Xiangzhe Xu, Zian Su, and Xiangyu Zhang. RepoAudit: An autonomous LLM-agent for repository-level code auditing. *arXiv preprint arXiv:2501.18160*, 2025.
- [17] Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, Xiaoheng Xie, and Xiangyu Zhang. LLM DFA: analyzing dataflow in code with large language models. *Advances in Neural Information Processing Systems*, 37:131545–131574, 2024.
- [18] Zhun Wang, Tianneng Shi, Jingxuan He, Matthew Cai, Jialin Zhang, and Dawn Song. CyberGym: Evaluating AI Agents’ Real-World Cybersecurity Capabilities at Scale. In *International Conference on Learning Representations*, 2026.
- [19] Andy K. Zhang, Joey Ji, Celeste Menders, Riya Dulepet, Thomas Qin, Ron Y. Wang, Junrong Wu, Kyleen Liao, Jiliang Li, Jinghan Hu, Sara Hong, Nardos Demilew, Shivatmica Murgai, Jason Khiem Tran, Nishka Kacheria, Ethan Jun-shen Ho, Denis Liu, Lauren McLane, Olivia Beyer Bruvik, Dai-Rong Han, Seungwoo Kim, Akhil Vyas, Cuiyuanxiu Chen, Ryan Li, Weiran Xu, Jonathan Z. Ye, Prerit Choudhary, Siddharth M. Bhatia, Vikram Sivashankar, Yuxuan Bao, Dawn Song, Dan Boneh, Daniel E. Ho, and Percy Liang. BountyBench: Dollar Impact of AI Agent Attackers and Defenders on Real-World Cybersecurity Systems. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2025.

- [20] Yuxuan Zhu, Antony Kellermann, Dylan Bowman, Philip Li, Akul Gupta, Adarsh Danda, Richard Fang, Conner Jensen, Eric Ihli, Jason Benn, Jet Geronimo, Avi Dhir, Sudhit Rao, Kaicheng Yu, Twm Stone, and Daniel Kang. CVE-Bench: A Benchmark for AI Agents’ Ability to Exploit Real-World Web Application Vulnerabilities. In *International Conference on Machine Learning*, 2025.
- [21] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambronero, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. Agentic bug reproduction for effective automated program repair at Google. *arXiv preprint arXiv:2502.01821*, 2025.
- [22] Kaibo Liu, Zhenpeng Chen, Yiyang Liu, Jie M Zhang, Mark Harman, Yudong Han, Yun Ma, Yihong Dong, Ge Li, and Gang Huang. LLM-powered test case generation for detecting bugs in plausible programs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 430–440, 2025.
- [23] Deniz Simsek, Aryaz Eghbali, and Michael Pradel. Pocgen: Generating proof-of-concept exploits for vulnerabilities in npm packages. *arXiv preprint arXiv:2506.04962*, 2025.
- [24] Wanzong Peng, Lin Ye, Xuetao Du, Hongli Zhang, Dongyang Zhan, Yunting Zhang, Yicheng Guo, and Chen Zhang. Pwngpt: Automatic exploit generation based on large language models. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2025, Vienna, Austria, July 27 - August 1, 2025, pages 11481–11494. Association for Computational Linguistics, 2025.
- [25] Zijie Zhao, Chenyuan Yang, Weidong Wang, Yihan Yang, Ziqi Zhang, and Lingming Zhang. AnyPoC: Universal proof-of-concept test generation for scalable LLM-based bug detection. *arXiv preprint arXiv:2604.11950*, 2026.
- [26] Xueying Du, Kai Yu, Chong Wang, Yi Zou, Wentai Deng, Zuoyu Ou, Xin Peng, Lingming Zhang, and Yiling Lou. Minimizing false positives in static bug detection via LLM-enhanced path feasibility analysis. *arXiv preprint arXiv:2506.10322*, 2025.
- [27] Derin Gezgün, Amartya Das, Shinhae Kim, Zhengdong Huang, Nevena Stojkovic, and Claire Wang. I can’t believe it’s not a valid exploit. *arXiv preprint arXiv:2602.04165*, 2026.
- [28] Minghao Shao, Sofija Jancheska, Meet Udeshi, Brendan Dolan-Gavitt, Haoran Xi, Kimberly Milner, Boyuan Chen, Max Yin, Siddharth Garg, Prashanth Krishnamurthy, Farshad Khorrami, Ramesh Karri, and Muhammad Shafique. NYU CTF Bench: A Scalable Open-Source Benchmark Dataset for Evaluating LLMs in Offensive Security. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [29] Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W. Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Jasper, Pura Peetathawatchai, Ari Glenn, Vikram Sivashankar, Daniel Zamoshchin, Leo Glikbarg, Derek Askaryar, Mike Yang, Teddy Zhang, Rishi Alluri, Nathan Tran, Rinnara Sangpisit, Polycarpus Yiorkadjis, Kenny Osele, Gautham Raghupathi, Dan Boneh, Daniel E. Ho, and Percy Liang. Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models. In *International Conference on Learning Representations*, 2025.
- [30] Xiaobao Wu, Liangming Pan, Yuxi Xie, Ruiwen Zhou, Shuai Zhao, Yubo Ma, Mingzhe Du, Rui Mao, Anh Tuan Luu, and William Yang Wang. Antileakbench: Preventing data contamination by automatically constructing benchmarks with updated real-world knowledge. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2025, Vienna, Austria, July 27 - August 1, 2025, pages 18403–18419. Association for Computational Linguistics, 2025.
- [31] Sophia Xiao Pu, Sitao Cheng, Xin Eric Wang, and William Yang Wang. Dynamic evaluation for oversensitivity in LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 2337–2344, 2025.

- [32] Michael Saxon, Ari Holtzman, Peter West, William Yang Wang, and Naomi Saphra. Benchmarks as microscopes: A call for model metrology. *arXiv preprint arXiv:2407.16711*, 2024.
- [33] Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. QL: Object-oriented queries on relational data. In *30th European Conference on Object-Oriented Programming, ECOOP 2016, LIPIcs*, pages 2:1–2:25. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016.
- [34] Giovanni Vigna. Autonomous vulnerability analysis, triaging, and repair: A historical perspective. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025*, page 1. ACM, 2025.
- [35] Vikram Nitin, Baishakhi Ray, and Roshanak Zilouchian Moghaddam. Faultline: Automated proof-of-vulnerability generation using llm agents. *arXiv preprint arXiv:2507.15241*, 2025.
- [36] Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. Otter: Generating tests from issues to validate swe patches. *arXiv preprint arXiv:2502.05368*, 2025.
- [37] Xiangpu Song, Longjia Pei, Jianliang Wu, Yingpei Zeng, Gaoshuo He, Chaoshun Zuo, Xiaofeng Liu, Qingchuan Zhao, and S Guo. Protocolguard: Detecting protocol non-compliance bugs via llm-guided static analysis and dynamic verification. In *Proceedings of the 33rd ISOC Network and Distributed System Security Symposium (NDSS)*, 2026.
- [38] Hwiwon Lee, Ziqi Zhang, Hanxiao Lu, and Lingming Zhang. SEC-bench: Automated Benchmarking of LLM Agents on Real-World Software Security Tasks. In *Advances in Neural Information Processing Systems*, 2025.
- [39] Longfei Chen, Ruibin Yan, Taiyu Wong, Yiyang Chen, Jialai Wang, and Chao Zhang. Smart-poc: Generating executable and validated pocs for smart contract bug reports. *arXiv preprint arXiv:2511.12993*, 2025.
- [40] Cen Zhang, Younggi Park, Fabian Fleischer, Yu-Fu Fu, Jiho Kim, Dongkwan Kim, Youngjoon Kim, Qingxiao Xu, Andrew Chin, Ze Sheng, Hanqing Zhao, Brian J. Lee, Joshua Wang, Michael Pelican, David J. Musliner, Jeff Huang, Jon Silliman, Mikel Mcdaniel, Jefferson Casavant, Isaac Goldthwaite, Nicholas Vidovich, Matthew Lehman, and Taesoo Kim. SoK: DARPA’s AI cyber challenge (AIXCC): Competition design, architectures, and lessons learned. *arXiv preprint arXiv:2602.07666*, 2026.
- [41] Google. From naptime to big sleep: Using large language models to catch vulnerabilities in real-world code. Project Zero Blog, 2024.
- [42] Nanda Rani, Kimberly Milner, Minghao Shao, Meet Udeshi, Haoran Xi, Venkata Sai Charan Putrevu, Saksham Aggarwal, Sandeep K. Shukla, Prashanth Krishnamurthy, Farshad Khorrami, Muhammad Shafique, and Ramesh Karri. CyberExplorer: Benchmarking LLM Offensive Security Capabilities in a Real-World Attacking Simulation Environment. *arXiv preprint arXiv:2602.08023*, 2026.
- [43] Nicholas Carlini, Edoardo Debenedetti, Javier Rando, Milad Nasr, and Florian Tramèr. AutoAdvExBench: Benchmarking Autonomous Exploitation of Adversarial Example Defenses. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 6778–6793. PMLR, 2025.
- [44] Luca Gioacchini, Alexander Delsanto, Idilio Drago, Marco Mellia, Giuseppe Siracusano, and Roberto Bifulco. AutoPenBench: A Vulnerability Testing Benchmark for Generative Agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1615–1624, Suzhou, China, 2025. Association for Computational Linguistics.
- [45] GitHub. CodeQL query help for Java. CodeQL documentation, 2026.

- [46] Benjamin Steenhoek, Hongyang Gao, and Wei Le. Dataflow analysis-inspired deep learning for efficient vulnerability detection. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*, pages 16:1–16:13. ACM, 2024.
- [47] Xin-Cheng Wen, Cuiyun Gao, Shuzheng Gao, Yang Xiao, and Michael R. Lyu. SCALE: constructing structured natural language comment trees for software vulnerability detection. In Maria Christakis and Michael Pradel, editors, *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, pages 235–247. ACM, 2024.
- [48] Claire Wang, Ziyang Li, Saikat Dutta, and Mayur Naik. QLCoder: A query synthesizer for static analysis of security vulnerabilities. *arXiv preprint arXiv:2511.08462*, 2025.
- [49] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. A C/C++ code vulnerability dataset with code changes and CVE summaries. In *MSR 2020: 17th International Conference on Mining Software Repositories*, pages 508–512. ACM, 2020.
- [50] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David A. Wagner, Baishakhi Ray, and Yizheng Chen. Vulnerability detection with code language models: How far are we? In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025*, pages 1729–1741. IEEE, 2025.

A Broader Impact

This work aims to advance the evaluation of AI coding agents on PoC generation in code-auditing scenarios. We have discussed several limitations of the current implementation in the main body of the paper. We do not expect our work to have negative broader impacts. The injected bugs are not existing vulnerabilities in deployed software systems, and therefore do not introduce new security threats to the open-source software ecosystem. We believe that our work has the potential to substantially influence future research on AI-assisted software security and code auditing, with broader implications for software auditing and defense.

More concretely, our work suggests several practical benchmark-construction insights for systematically adjusting PoC-generation difficulty. The evaluation results show that execution context length, transformation depth, and bug type diversity all substantially affect PoC-generation performance across different coding agents. These dimensions provide practical mechanisms for constructing benchmark cases with different reasoning requirements and difficulty levels. This is important for future automated construction of PoC-generation benchmarks, as it can provide evaluation and training data that simulate different levels of difficulty for existing PoC-generation techniques.

Although our current implementation focuses on Maven-based Java projects and executable bug types derived from CodeQL Java queries, the underlying methodology is not intrinsically tied to Java or Maven. More generally, the same workflow can potentially be extended to other programming languages and software ecosystems with executable test environments and verifiable validation signals. We hope this work can encourage future research on scalable, execution-validated evaluation methodologies for AI-assisted software engineering and security analysis.

B Implementation Details

Project and Bug Type Preparation. We instantiate POCGYM on six real-world Java projects spanning distributed middleware, data serialization, and multipurpose utility libraries. These projects provide diverse and realistic injection sites together with varied execution contexts. All selected projects use Maven as their build system, which allows an individual test case to be re-played with a localized command such as:

```
./mvnw -q -pl <module> -Dtest=<test-class> test.
```

We exclude tests that depend on external services, network access, nondeterministic timing, or excessive runtime. Each experiment starts from a clean repository template, and all injection, transformation, and evaluation runs are performed in isolated working copies. Our current implementation targets Maven-based Java projects. However, POCGYM is not intrinsically tied to Java or Maven. The same workflow can be extended to other languages and build systems, provided that the build system supports isolated execution of a designated test case, which is necessary for collecting execution contexts.

For bug type selection, we require the injected defect to produce an observable failure, such as an assertion failure or a runtime exception. Types whose effects are difficult to observe deterministically, such as pure performance regressions, are outside the scope of this benchmark. Based on this criterion, we select ten bug types grounded in the CodeQL Java query families for benchmark construction. Representative types include Guard Predicate Inversion, Intra-Function Null Dereference and Comparison of Identical Values. Table 5 lists all ten types organized by category.

Execution Context Collection. For each selected test case, we collect the ordered production-side execution path exercised by the test and summarize it as its execution context. To study how execution context length affects PoC generation, we stratify the collected execution contexts into length-based buckets. Here, execution context length refers to the number of distinct production-side functions reached during the test run, as recorded in the execution log τ_t . Specifically, we use four buckets: *short*, *mid-short*, *mid-long*, and *long*, with execution context lengths of $[1, 40]$, $[41, 70]$, $[71, 120]$, and $[121, 150]$ production functions, respectively. Shorter contexts correspond to more constrained execution paths. Longer contexts capture broader execution paths that may traverse multiple production functions, classes, or modules.

Table 5: The ten bug types used in the benchmark construction.

Category	Type	Bug Type description
API Contract	Comparison of Identical Values	A condition compares a value with itself, producing a constant result that silently disables validation or equality logic.
	Typo in <code>equals</code>	A function overriding <code>equals(Object)</code> has the wrong name or signature, so Java keeps using reference equality.
	Typo in <code>hashCode</code>	A function intended to override <code>hashCode()</code> has the wrong name or signature, so hashed collections use identity hashing.
	Hashed Value Without <code>hashCode</code>	A class overrides <code>equals(Object)</code> but not <code>hashCode()</code> , so logically equal keys may hash to different buckets.
	Inconsistent <code>equals</code> and <code>hashCode</code>	<code>equals</code> and <code>hashCode</code> use different fields or unstable state, breaking lookup in <code>HashMap</code> or <code>HashSet</code> .
Value Flow	Intra-Function Null Dereference	A value that was checked for null is later overwritten, moved outside the guard, or dereferenced on an unguarded path.
	Collection Element Null Dereference	Maintenance or eviction leaves a null element in a list, map, or cache, and a later lookup or iteration assumes it is non-null.
Logic	Guard Predicate Inversion	A guard condition is flipped or weakened, sending valid inputs to an error or skip path, or letting invalid inputs continue.
	Skipped Aggregation Counter	A branch still processes an item but skips the counter or total update, so returned metrics undercount the work performed.
	In-Bounds Collection Misrouting	A loop writes an element to a legal but wrong index or key, leaving the collection complete but logically misordered.

Buggy Code Injection. We perform bug injection with Claude Code backed by Claude Sonnet 4.5. For each candidate case, the injection process is allowed up to three attempts. Before every attempt, the repository is restored to a clean workspace so that failed attempts do not accumulate state. A candidate is archived only if it satisfies three sequential validation checks. First, it must introduce a genuine production-code diff; cases with no change, formatting-only edits, or modifications confined to test or build files are rejected. Second, the diff must be structurally consistent with the intended bug type; we use lightweight syntactic checks to filter out obvious mismatches, such as injecting a null-pointer defect when the requested type is an equality inconsistency. Third, the replay command must pass on the clean baseline and expose the injected bug on the modified repository; cases that fail to compile, fail for unrelated infrastructure reasons, or do not trigger the intended defect are discarded.

Bug-Preserving Transformation. After obtaining a valid injected case, we apply bug-preserving transformations, also using Claude Code backed by Claude Sonnet 4.5. Each transformation process allowed one attempt. To diversify the structural embedding of injected bugs, we define ten transformation operators across three families. Table 6 lists all ten operators. Each transformation operator is instantiated through a prompt that instructs the agent to rewrite the code region containing the injected bug while preserving its triggerability. We apply transformations at four depths: 1, 3, 5, and 10. The depth denotes the number of transformation operators applied sequentially after the initial injection. Each transformation step is validated immediately by rebuilding the project and rerunning the replay command. A transformed case is retained only if every applied transformation preserves triggerability: the modified project must remain valid, and the same test case must still expose the injected bug. This validation ensures that transformations increase structural variation without repairing the defect.

Table 6: The ten bug-preserving transformation operators.

Family	Operator	Effect
Call Stack	Helper Call Insertion	Adds an internal helper or wrapper call between the original caller and the defect site, increasing call-chain depth.
	Call-Site Rerouting	Replaces a direct internal call with a forwarding function, so the same logic is reached through an extra dispatch step.
	Extract-and-Apply Split	Splits one computation into an extraction helper and an application helper, forcing the defect to cross function boundaries.
Data Flow	Parameter Bundling	Packs related arguments or locals into an internal object that is passed through helper calls instead of using separate values.
	Local State Bundling	Moves related local variables into a context object, so later code reads state through fields rather than direct locals.
	Intermediate State Passing	Stores intermediate values in a state holder and reads them later across helper calls, lengthening the value flow path.
	Cached Value Access	Routes reads of a derived value through a cached accessor instead of direct recomputation or direct field access.
	Nested Data Access	Replaces direct value access with access through a nested container, wrapper, map entry, or list element.
	Pipeline Stage Split	Breaks a multi-step function into ordered internal stages, making the defect appear inside a staged processing pipeline.
Control Flow	Rare Input Path Conditioning	Restricts the defect path to rare but deterministic input conditions, preserving normal behavior for common inputs.

C Detailed Statistics of Benchmark

The constructed benchmark contains 1,300 archived buggy cases from six real-world Java projects. Each case stores a buggy repository snapshot, the execution context associated with the selected test, the applied bug type, the applied transformation sequence, and the reference PoC used for validation. Figure 5 summarizes the benchmark composition. At the case level, the benchmark contains 201 execution contexts, distributed across four execution context length buckets: 56 short contexts, 45 mid-short contexts, 45 mid-long contexts, and 55 long contexts.

The benchmark covers all ten bug types in Table 5. Guard Predicate Inversion appears most frequently, with 688 cases, because it is used as the anchor type for systematically varying transformation depth and execution context length. The remaining nine types each appear in 68 cases, yielding balanced coverage over the other Logic, Value Flow, and API Contract defects. At the category level, the benchmark therefore contains 824 Logic cases, 136 Value Flow cases, and 340 API Contract cases. Across all cases, the ten transformation operators in Table 6 appear 4,406 times in total. The most frequent operators are Call-Site Rerouting (1,028 occurrences), Helper Call Insertion (1,026), Local State Bundling (801), and Rare Input Path Conditioning (609), followed by Intermediate State Passing (218), Parameter Bundling (158), Cached Value Access (151), Nested Data Access (146), Pipeline Stage Split (141), and Extract-and-Apply Split (128).

Overall, the benchmark construction successfully achieved approximately 63% of attempted candidates. This cost estimate includes failed attempts as well as the successful bug injection, transformation, execution context collection, replay validation, and reference-PoC validation runs. The end-to-end construction cost is approximately \$16,500.

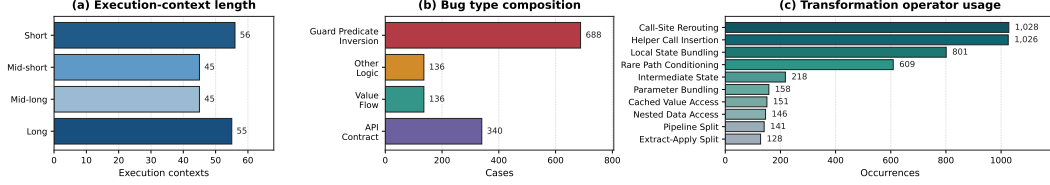


Figure 5: The distributions of execution context length, different categories of bug types, and the occurrence of transformation operators.

Table 7: PoC-generation success rates across execution context length buckets. The subscript annotations indicate the difference between each success rate and the corresponding rate in the short bucket.

Agent ID	Short		Mid-Short		Mid-Long		Long	
	NoEC	WithEC	NoEC	WithEC	NoEC	WithEC	NoEC	WithEC
A1	80%	80%	72% ↓8pp	76% ↓4pp	64% ↓16pp	64% ↓16pp	48% ↓32pp	52% ↓28pp
A2	72%	76%	64% ↓8pp	68% ↓8pp	56% ↓16pp	56% ↓20pp	40% ↓32pp	40% ↓36pp
A3	60%	64%	52% ↓8pp	56% ↓8pp	40% ↓20pp	44% ↓20pp	28% ↓32pp	28% ↓36pp
A4	60%	60%	52% ↓8pp	56% ↓4pp	40% ↓20pp	40% ↓20pp	28% ↓32pp	28% ↓32pp
A5	44%	44%	36% ↓8pp	40% ↓4pp	28% ↓16pp	28% ↓16pp	16% ↓28pp	20% ↓24pp
A6	24%	28%	20% ↓4pp	20% ↓8pp	12% ↓12pp	16% ↓12pp	8% ↓16pp	8% ↓20pp

D Extended Analysis of Execution Context Effect

This appendix expands RQ3 using the same 400-case controlled subset of Guard Predicate Inversion used in the main text. The subset contains four execution context length buckets and four transformation depth buckets, with 25 validated cases for each bucket combination. Table 7 groups these cases by execution context length and aggregates over transformation depths.

Table 7 provides a more fine-grained view of how execution context length affects PoC generation. The baseline-relative annotations show that the degradation is not limited to a few agents or a single evaluation setting. Instead, all evaluated agents experience lower success rates as the execution context becomes longer under both NoEC and WithEC. For example, under WithEC, Codex with GPT-5.4 (A1) drops from 80% in the short bucket to 52% in the long bucket, while Claude Code with Claude Sonnet 4.5 (A2) drops from 76% to 40%. The same pattern also appears for open-source agent frameworks with open-weight models, where OpenHands with DeepSeek-V3.2 (A3) decreases from 64% to 28%, and OpenCode with ZAI GLM-4.7 (A4) decreases from 60% to 28%.

The table also shows that execution context information does not fully offset the difficulty introduced by longer contexts. In several buckets, WithEC improves over NoEC, suggesting that context guidance can help agents identify potentially relevant functions and program regions. However, the improvement is not sufficient to prevent the global decline. For instance, A1 still loses 28 percentage points under WithEC when moving from short to long execution contexts, and A2 loses 36 percentage points. This indicates that the main challenge is not only finding where the bug may be triggered, but also converting a longer execution context into a concrete executable test. As the context grows, the agent must select relevant setup operations, preserve intermediate program states, and construct an oracle that exposes the defect without being distracted by plausible but irrelevant call paths.

Another observation is that the degradation is often progressive across buckets. Many agents lose performance already when moving from short to mid-short contexts, and the loss becomes more pronounced in mid-long and long contexts. This suggests that execution context length acts as a trigger-side complexity factor: it expands the amount of program behavior that the PoC must reconstruct before the bug becomes observable. Therefore, even when execution context information is available, longer context still imposes substantial synthesis difficulty on current coding agents.

Table 8: PoC-generation success rates across transformation depth buckets. The subscript annotations indicate the difference between each success rate and the corresponding rate at depth 1.

Agent ID	Depth 1		Depth 3		Depth 5		Depth 10	
	NoEC	WithEC	NoEC	WithEC	NoEC	WithEC	NoEC	WithEC
A1	76%	80%	67% ↓9pp	68% ↓12pp	62% ↓14pp	65% ↓15pp	59% ↓17pp	59% ↓21pp
A2	70%	72%	59% ↓11pp	60% ↓12pp	53% ↓17pp	57% ↓15pp	50% ↓20pp	51% ↓21pp
A3	57%	60%	45% ↓12pp	49% ↓11pp	42% ↓15pp	43% ↓17pp	36% ↓21pp	40% ↓20pp
A4	53%	55%	47% ↓6pp	48% ↓7pp	41% ↓12pp	41% ↓14pp	39% ↓14pp	40% ↓15pp
A5	38%	40%	33% ↓5pp	32% ↓8pp	28% ↓10pp	32% ↓8pp	25% ↓13pp	28% ↓12pp
A6	22%	23%	17% ↓5pp	16% ↓7pp	12% ↓10pp	16% ↓7pp	13% ↓9pp	17% ↓6pp

E Extended Analysis of Transformation Depth Effect

This appendix expands RQ4 using the same 400-case controlled subset. The subset contains four transformation depth buckets and four execution context length buckets, with 25 validated cases for each bucket combination. Table 8 groups these cases by transformation depth and aggregates over execution context lengths.

Table 8 further expands the analysis of transformation depth. Across both NoEC and WithEC, deeper bug-preserving transformations generally reduce PoC-generation success rates. Under WithEC, Codex with GPT-5.4 (A1) decreases from 80% at depth 1 to 59% at depth 10, and Claude Code with Claude Sonnet 4.5 (A2) decreases from 72% to 51%. The same trend appears for open-source agents: OpenHands with DeepSeek-V3.2 (A3) decreases from 60% to 40%, and OpenCode with ZAI GLM-4.7 (A4) decreases from 55% to 40%. These results support the main observation in RQ4 that transformation depth can serve as a practical dimension for adjusting PoC-generation difficulty.

The effect of transformation depth differs from the effect of execution context length. Execution context length increases the amount of triggering behavior that the test must reconstruct, whereas transformation depth changes how the injected bug is structurally embedded in the code. As more transformations are applied, the same underlying bug may be surrounded by additional code fragments, modified control structure, or altered data dependencies. Consequently, the agent must reason through a transformed implementation before it can synthesize a valid setup and oracle. This makes transformation depth a construction-side complexity factor that complements execution context length.

At the same time, transformation depth should not be interpreted as a perfectly monotonic difficulty scale. The table contains small plateaus and reversals for some lower-performing agents. For example, under WithEC, OpenHands with Devstral-2-123B (A5) remains at 32% from depth 3 to depth 5, and OpenCode with Qwen3-Coder-480B-A35B-Instruct (A6) increases from 16% at depth 5 to 17% at depth 10. Such minor irregularities are expected because transformations are compositional: each additional transformation can affect code shape, control flow, and data flow in different ways. Therefore, the number of transformations provides a useful coarse-grained difficulty control, but it does not precisely determine the semantic complexity of every generated case. Designing more fine-grained transformation metrics that account for the concrete structural and semantic effects of individual operators is an important direction for future benchmark construction.

F Additional Analysis of Transformation Dimension Effects

Operator-level effects. We first analyze PoC-generation success at the transformation-operator level. We use the same 400-case transformation depth subset as in RQ4. Because a transformed case may contain multiple operators, this analysis aggregates operator occurrences rather than treating operators as mutually exclusive case groups.

As shown by Table 9, the operator-level effect is smaller than the effects of execution context length and transformation depth, but it is still diagnostic. Rare Input Path Conditioning has the highest average WithEC success rate (57.6%), followed by Cached Value Access (51.7%). These operators usually preserve a relatively direct relationship between the triggering input and the observed failure,

Table 9: Transformation-operator success rates by agents. Each entry reports WithEC/NoEC in percent.

Family	Transformation Operator	A1	A2	A3	A4	A5	A6	Mean
Call Stack	Helper Call Insertion	63.5/60.2	52.3/49.0	41.1/39.0	42.3/37.8	25.7/23.2	8.3/7.5	38.9/36.1
	Call-Site Rerouting	70.0/69.6	55.7/56.1	48.9/45.6	48.5/45.6	38.4/29.1	20.7/17.7	47.0/44.0
	Extract-and-Apply Split	63.1/61.2	49.5/50.5	36.9/37.9	35.9/37.9	21.4/22.3	9.7/9.7	36.1/36.6
Data Flow	Parameter Bundling	63.9/60.3	60.3/55.2	44.8/41.8	42.8/39.7	32.0/27.8	16.0/12.9	43.3/39.6
	Local State Bundling	63.6/63.6	58.3/60.4	43.3/43.9	37.4/40.1	29.4/28.3	14.4/11.2	41.1/41.3
	Intermediate State Passing	61.3/59.5	51.8/48.8	35.1/32.1	36.3/36.3	25.0/23.8	12.5/8.9	37.0/34.9
	Cached Value Access	75.5/73.0	67.5/66.9	53.4/50.9	51.5/54.0	42.3/42.3	20.2/22.1	51.7/51.5
	Nested Data Access	65.6/65.6	60.3/54.3	48.3/43.7	43.0/45.7	28.5/28.5	13.9/13.9	43.3/41.9
	Pipeline Stage Split	68.3/62.7	61.9/61.1	53.2/47.6	51.6/51.6	32.5/34.1	15.1/17.5	47.1/45.8
Control Flow	Rare Input Path Conditioning	77.0/75.2	72.4/69.4	62.1/57.0	58.2/55.2	42.4/42.1	33.6/28.5	57.6/54.5

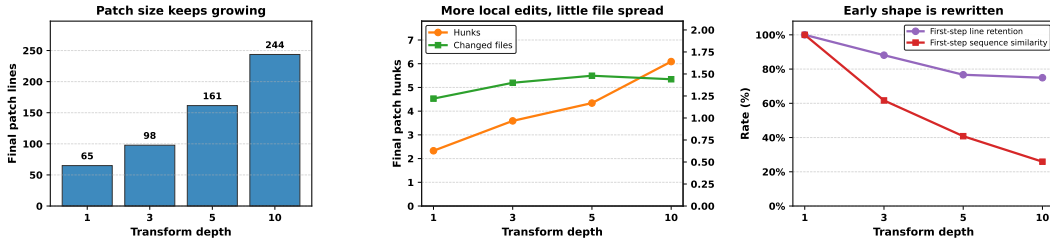


Figure 6: Inject-stage patch evolution by transformation depth. Deeper transformations increase final patch size and hunk count, while the number of changed files remains nearly flat.

so the generated PoC mainly needs to reach the right condition or accessor path. In contrast, the lowest-success operators are Extract-and-Apply Split (36.1%), Intermediate State Passing (37.0%), and Helper Call Insertion (38.9%). These operators introduce helper boundaries or intermediate state, making the generated test more sensitive to setup order and oracle placement.

The benefit of execution context guidance is also operator-dependent. Parameter Bundling (+3.7 pp), Call-Site Rerouting (+3.1 pp), and Rare Input Path Conditioning (+3.1 pp) benefit most from execution context, suggesting that path information helps when an operator mainly changes the route to the defect. In contrast, Local State Bundling is nearly unchanged (-0.2 pp), and Extract-and-Apply Split is slightly worse under WithEC (-0.5 pp). This suggests that execution context guidance is less decisive when the main difficulty lies in reconstructing repackaged state or placing the oracle after split helper logic.

Inject-stage patch-evolution effects. The preceding analysis focuses on whether agents can generate executable PoCs. We further analyze the inject-stage artifacts to examine how transformation depth changes the structure of the generated buggy patches. This analysis uses the same 400-case transformation depth subset as RQ4, with one fixed bug type, four transformation depth buckets, and 100 successfully validated injected cases per bucket. Across these cases, we analyze 1,900 step-level transformation patches and the final buggy patch retained for each case.

Figure 6 and Table 10 show that transformation depth increases final patch size and fragmentation without substantially spreading edits across more files. The mean final patch size grows from 65.0 non-empty diff lines at depth 1 to 243.7 at depth 10, while the mean hunk count grows from 2.33 to 6.09. In contrast, the mean number of changed files remains nearly flat, from 1.22 to 1.44. Thus, deeper bug-preserving transformations mainly create local structural indirection inside a small number of files, rather than diffusing the injected bug across the repository.

Transformation depth also changes how earlier transformation steps survive in the final buggy patch. First-step added-line retention remains relatively high, dropping from 100.0% at depth 1 to 74.9% at depth 10. However, first-step sequence similarity drops much more sharply, from 100.0% to 25.9%. This means that early transformation content often remains partially present, but its original patch shape is wrapped, split, moved, or rewritten by later operators. The line-accumulation factor supports the same interpretation: it rises from $1.00\times$ at depth 1 to $6.63\times$ at depth 10, showing that

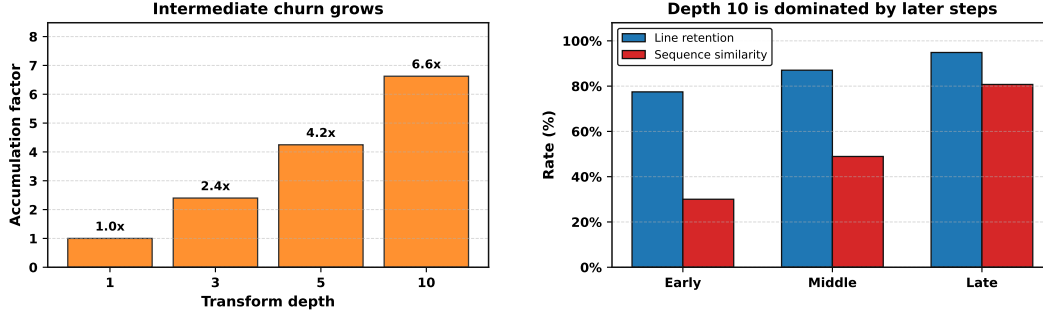


Figure 7: Intermediate edit churn and step-position effects. At depth 10, later transformation steps dominate the final buggy patch shape more strongly than earlier steps.

Table 10: Inject-stage patch-evolution metrics by transformation depth. Line accumulation is the ratio between cumulative step-patch lines and final-patch lines. Retention and sequence similarity are measured from the first transformation step to the final buggy patch.

Depth	Runs	Final lines	Hunks	Changed files	Line accum.	First-step retention	First-step similarity
1	100	65.0	2.33	1.22	1.00×	100.0%	100.0%
3	100	97.8	3.59	1.40	2.40×	88.1%	61.7%
5	100	161.5	4.34	1.48	4.25×	76.7%	40.8%
10	100	243.7	6.09	1.44	6.63×	74.9%	25.9%

deeper transformation chains create substantial intermediate edit churn rather than simple additive stacking.

Figure 7 further shows that, at depth 10, later transformation steps dominate the final patch shape. Early steps retain 77.5% of added lines on average, but only 30.0% sequence similarity, while late steps reach 94.9% line retention and 80.7% sequence similarity. This supports a more precise interpretation of transformation depth: later operators do not completely erase the semantics introduced by earlier transformations, but they increasingly absorb and reshape earlier structure. Together, these results explain why transformation depth can increase PoC-generation difficulty: it makes the final buggy patch larger and less recognizable while keeping the injected changes locally concentrated.

G Prompt Design

We report the prompt structures used in PoCGYM. The implementation uses layered prompts rather than a single monolithic template: a reusable system prompt specifies global behavioral constraints, while a dynamically assembled task payload injects case-specific fields such as the bug type, bug type specification, transformation operator, execution context, replay command, candidate anchors, and file scope. For readability, repository-local paths and long execution context lists are shortened in this appendix, but the task goals, constraints, input fields, and required output formats follow the actual pipeline. The implementation sometimes uses the term *trigger generation*; in paper-facing terminology, this corresponds to PoC generation.

G.1 Bug Injection System Prompt

Bug Injection System Prompt

Mission. Introduce a real, localized bug into production code so that the externally provided replay command fails for the intended bug type. The repository must end with a non-empty semantic git diff, and the replay command must fail because of the injected bug.

Core rules.

1. Produce a non-empty semantic production-code diff.
2. Run the replay command exactly as provided, without changing the module, flags, test target, or command structure.

Table 11: Layered prompt structures used by PoCGYM during benchmark construction and PoC-generation evaluation.

Prompt Structure	Main Inputs	Required Output
Bug Injection System Prompt	Global injection rules, replay-command discipline, bounded edit-run-revise loop, diff checks, and success criteria	Single-line INJECT_REPORT_JSON with operational evidence fields
Bug Injection Task Payload	Bug type, bug type specification, required and forbidden elements, execution context, candidate anchors, trigger hints, and file scope	Same INJECT_REPORT_JSON prefix with type-site rationale fields
Bug-Preserving Transformation Task Payload	Transformation operator specification, applicability requirements, preservation constraints, execution context, and candidate anchors	Single-line TRANSFORM_REPORT_JSON
PoC-Generation System Prompt	Test-only edit constraints, Java testcase materialization rules, PoC-only objective, and evaluation-attempt policy	Single-line TRIGGER_REPORT_JSON fallback schema
PoC-Generation Task Payload	Buggy repository metadata, case ID, bug type, applied transformations, bug-related files, replay command, and optional execution context	Single-line TRIGGER_REPORT_JSON with detailed PoC-generation fields

3. Do not modify tests unless explicitly allowed by the task.
4. Do not modify build files, dependency files, CI files, or repository configuration.
5. Do not change public or protected API signatures.
6. Keep the edit localized, realistic, and consistent with the repository style.
7. Do not claim success if the git diff is empty or if the observed failure is unrelated to the injected bug.

Construction-side execution budget. During one bug-injection session, the implementation permits at most three runs of the provided replay command. If the replay command still passes, the agent must revise the production-code edit and rerun the same command within this budget. This construction-side replay budget is separate from the PoC-generation attempt budget used in the evaluation.

Required final output. The system prompt requires exactly one final single-line JSON object prefixed by INJECT_REPORT_JSON. The operational evidence fields include: `ok`, `git_changed_files`, `repro_cmd`, `repro_exit_code`, `observed_exception`, `call_chain`, and `notes`.

G.2 Bug Injection Task Payload

Bug Injection Task Payload

Task-specific inputs.

- Bug type and bug type ID.
- Bug type name, category, scope, and language.
- Bug type description.
- Required elements that must appear in the patch.
- Forbidden elements that must not appear in the patch.
- Expected failure signal and trigger-condition hints.
- Execution context summary collected from the target test.
- Candidate anchors with trace-relevant methods, files, and candidate edit sites.
- Allowed or preferred production-code file scope.

Bug type contract. The injected bug must match the provided bug type specification rather than merely resemble it. Required elements must be visible in the actual patch, and forbidden elements must not appear in the final patch. The chosen site should be trace-relevant and should allow the provided replay command to trigger the injected bug after validation.

Task-level reporting fields. The task payload uses the same INJECT_REPORT_JSON prefix and adds bug-type-site rationale fields: `ok`, `type_id`, `target_files`, `target_symbols`, `selected_anchor`, `why_this_site`, `expected_trigger_path`, `stealth_rationale`, `bug_mechanism`, `preserved_prior_structure`, and `possible_risks`. The implementation field `stealth_rationale` records why the edit is localized and review-plausible; in paper-facing terminology, this supports realistic bug injection and data-contamination mitigation rather than a separate evaluation objective.

G.3 Bug-Preserving Transformation Task Payload

Bug-Preserving Transformation Task Payload

The transformation prompt is a dynamically assembled task payload generated from an internal transformation operator specification. The payload uses implementation-level fields such as `id`, `name`, `intent`, `description`, `applicability`, `forbidden_elements`, `must_preserve`, and `post_conditions`. Each internal operator is mapped to a paper-facing transformation operator and family. For example, `t_rare_profile_gate_v1` is reported as Rare Input Path Conditioning, `t_call_stack_deepen_v1` is reported as Helper Call Insertion, and `t_private_adapter_layer_v1` is reported as a call-site rerouting / private-adapter transformation.

Task. Apply one bug-preserving transformation step after successful bug injection. The transformation should alter the surrounding production-code structure while preserving compilation and preserving the replay behavior that exposes the injected bug. It is bug-preserving in the benchmark-construction sense: it preserves the relevant trigger path and bug-inducing behavior, but it is not required to preserve the full behavior of the original unmodified program.

Inputs.

- Internal transformation ID and operator name.
- Paper-facing operator term and transformation family.
- Operator intent and description.
- Applicability requirements.
- Forbidden edits.
- Must-preserve constraints, including compilation success, public API signatures, behavior for non-trigger inputs when applicable, and replay-command triggerability.
- Post-conditions expected from the operator.
- Execution context summary and candidate anchors.

Edit boundaries.

1. Do not modify tests unless explicitly allowed by the operator.
2. Do not change public or protected API signatures.
3. Do not perform broad refactors, renames, or formatting sweeps.
4. Keep the transformation localized to trace-relevant production code.
5. Preserve the injected bug-triggering behavior under the same replay command.

Required final output. The payload requires exactly one final single-line JSON object prefixed by `TRANSFORM_REPORT_JSON`. The JSON object contains: `ok`, `transform_id`, `changed_files`, `edit_sites`, `what_was_added_or_wrapped`, `what_was_preserved`, `what_was_replaced_or_overwritten`, `compatibility_with_previous_steps`, `expected_effect_on_stealth`, and `expected_effect_on_trigger_depth`. The field `expected_effect_on_stealth` is the implementation field name for the expected surface-form and review-plausibility effect of the transformation.

G.4 PoC-Generation System Prompt

PoC-Generation System Prompt

Mission. Create a Java PoC test that reproduces an already-injected production bug under the provided Maven replay command. The agent's job is not to fix code, improve the repository, or write a generic regression test. The only goal is to materialize a test that exposes the injected bug.

Evaluation attempt budget. For each benchmark case, the evaluation protocol allows at most three PoC-generation attempts. Each attempt must follow the same test-only constraints and use the provided replay command as the validation target. A run is counted as successful only if the generated PoC test compiles, the replay command exposes the injected defect, and no production file or build configuration is modified.

Hard constraints.

1. Never modify production code.
2. Never modify build files, dependency files, CI files, configuration files, or repository metadata.
3. Only create or edit Java test files under `src/test/java`.
4. Prefer rewriting the official test file at the provided path and with the provided package and class name.
5. The PoC test must materialize in the repository as a real `.java` file before verification.
6. The test must be designed to trigger the injected bug, not to validate normal behavior.
7. Do not delete or modify unrelated files.

Required final output. The system prompt requires a final single-line `TRIGGER_REPORT_JSON`. The fallback schema includes: `ok`, `test_strategy`, `target_bug_path_hypothesis`, `why_trigger_should_fail`, and `materialized_test_files`.

G.5 PoC-Generation Task Payload

PoC-Generation Task Payload

Task-specific inputs.

- Repository name.
- Evaluation group or context condition.
- Injected case run ID.
- Bug type ID and bug type.
- Applied bug-preserving transformations.
- Bug-related production files and location metadata.
- Injection-stage verification exit code.
- Maven replay command used for validation.
- Natural-language bug summary.
- Target test path and target test class when available.

Execution context handling. The two evaluation settings are NoEC and WithEC. In WithEC, the task payload includes the execution context summary collected during benchmark construction. In NoEC, this execution context block is omitted. Neither setting provides the reference PoC contents, hidden validation oracle, construction-time solution code, or permission to modify production code.

Required final output. The task payload asks for a detailed final TRIGGER_REPORT_JSON schema with: summary, suspected_bug_mechanism, inspected_production_files, inspected_test_files, selected_test_path, selected_test_class, trigger_strategy, expected_failure_signal, materialized_test_files, and notes.

G.6 End-to-End Case Prompt Example

This example shows a concrete case-level prompt flow assembled by the pipeline. The case uses repository dubbo, bug type Guard Predicate Inversion, and a depth-3 transformation chain. Long execution context lists are shortened for page fit, but the fields and constraints shown below follow the actual prompt structure.

Case Metadata

```
run_id: inject__20260315T064448Z
repository: dubbo
trace_group: short
trace_id: org.apache.dubbo.common.url.URLParamTest
bug_type: Guard Predicate Inversion
type_id: logic_predicate_inversion_in_guard
transform_depth: 3
transform_ids:
  - t_rare_profile_gate_v1
  - t_call_stack_deepen_v1
  - t_private_adapter_layer_v1
paper_facing_transform_terms:
  - Rare Input Path Conditioning
  - Helper Call Insertion
  - Call-Site Rerouting / Private Adapter Layer
official_test_path: dubbo-common/src/test/java/org/apache/dubbo/common/url/URLParamTest.java
official_test_class: org.apache.dubbo.common.url.URLParamTest
replay_command: ./mvnw -q -pl dubbo-common -Dtest=org.apache.dubbo.common.url.URLParamTest test
context_setting_for_poc_generation: WithEC
evaluation_attempt_budget: at most 3 PoC-generation attempts per benchmark case
```

Bug Injection Prompt

SYSTEM:

You are operating inside a Git repository in an automated bug-injection pipeline.

MISSION:

Introduce a real, localized bug into production code so that the externally provided reproduction command fails for the intended bug type. Your work is only valid if the repository ends the session with a non-empty semantic git diff and the provided reproduction command, run verbatim, fails because of the injected bug.

NON-NEGOTIABLE RULES:

- You must produce a non-empty semantic git diff in the current repository state before finalizing.
- Semantic git diff means a real logic-changing edit, not whitespace-only, formatting-only, comment-only, import-only, rename-only, or mechanically trivial changes.
- You must run the reproduction command exactly as provided in the task, verbatim.
- Do not invent, replace, shorten, simplify, or substitute any command, module, flag, path, or test target.
- Do not modify test code unless the task explicitly allows it.
- Do not change public or protected API signatures.
- Do not claim success based on a hypothetical edit.
- Do not output a final success JSON if git diff is empty.
- Do not output a final success JSON if the reproduction command in your JSON differs from the externally provided command.

CONSTRUCTION-SIDE EXECUTION BUDGET:

- Up to 3 total runs of the provided reproduction command in the same injection session.
- If the reproduction command passes, make another production-code edit and rerun the same command verbatim within this budget.

TASK PAYLOAD:

You are performing controlled BUG INJECTION into production code for research benchmarking.

Goal: introduce a subtle, realistic bug consistent with the specified bug type, while keeping changes localized and review-plausible.

TYPE:

- id: logic_predicate_inversion_in_guard
- name: Predicate inversion in critical guard
- category: Logic
- scope: intra_method_branch_condition
- language: java

Description:

A boolean predicate used to guard important logic is inverted or altered in a subtle way. The code still compiles, branches look plausible, but the semantics flip which path handles which inputs, causing assertions and invariants to fail.

Required elements:

- if_or_else_block_guarding_non_trivial_logic
- condition_based_on_collection_size_or_status_flag
- tests_that_distinguish_between_valid_and_invalid_input_cases

Forbidden elements:

- conditions_that_are_constant_true_or_false_after_modification
- completely_removing_the_guard_and_inlining_the_body
- branches_with_no_observable_effect

Runtime effect hints:

- exception_type_hint: none
- typical_failure_signal: assertion failure or invalid return value

Trigger condition hints:

- test_shape_hint: tests feed different valid/invalid input categories and assert different behavior
- input_properties_hint: at least one test relies on the original guard semantics

TRACE CONTEXT SUMMARY:

```
{
  "trace_id": "org.apache.dubbo.common.url.URLParamTest",
  "test_name": "org.apache.dubbo.common.url.URLParamTest",
  "methods_count": 91
}
```

CANDIDATE ANCHORS:

- method: URLParamTest
file: dubbo-common/src/test/java/org/apache/dubbo/common/url/URLParamTest.java
role: target test
- method: hasMethodParameter
file: dubbo-common/src/main/java/org/apache/dubbo/common/url/component/URLParam.java
role: trace-relevant guard candidate
- method: getMethodParameter
file: dubbo-common/src/main/java/org/apache/dubbo/common/url/component/URLParam.java
role: trace-relevant lookup candidate
- method: getMethodParameterStrict
file: dubbo-common/src/main/java/org/apache/dubbo/common/url/component/URLParam.java
role: trace-relevant lookup candidate
- method: initMethodParameters
file: dubbo-common/src/main/java/org/apache/dubbo/common/url/component/URLParam.java
role: method-parameter initialization path
- method: size
file: dubbo-common/src/main/java/org/apache/dubbo/common/url/component/URLParam.java

```

role: status/collection-size related helper
- method: isEmpty
file: dubbo-common/src/main/java/org/apache/dubbo/common/url/component/URLParam.java
role: status/collection-size related helper

```

DELIVERABLE:

1. Implement the bug strictly according to the bug type contract.
2. Keep the patch minimal and consistent with repository style.
3. Run the provided reproduction command verbatim.
4. Provide evidence in the final JSON.

REQUIRED FINAL OUTPUT:

Append exactly one final line with this prefix:

```
INJECT_REPORT_JSON: { ... }
```

Required field groups:

- operational_evidence: ok, git_changed_files, repro_cmd, repro_exit_code, observed_exception, call_chain, notes
- type-site_rationale: type_id, target_files, target_symbols, selected_anchor, why_this_site, expected_trigger_path, stealth_rationale, bug_mechanism, preserved_prior_structure, possible_risks

Bug Injection Report

INJECT_REPORT_JSON:

```

{
  "ok": true,
  "type_id": "logic_predicate_inversion_in_guard",
  "target_files": [
    "dubbo-common/src/main/java/org/apache/dubbo/common/url/component/URLParam.java"
  ],
  "target_symbols": [
    "getAnyMethodParameter"
  ],
  "selected_anchor": "getAnyMethodParameter",
  "why_this_site": "The method is reached by URLParamTest and contains a critical guard over method-specific parameter lookup.",
  "expected_trigger_path": "URLParamTest -> URLParam.getAnyMethodParameter -> inverted guard returns null instead of the expected method parameter.",
  "stealth_rationale": "The edit is a localized predicate substitution inside an existing defensive check.",
  "bug_mechanism": "A guard condition is inverted from a non-empty map check to an empty map check, swapping valid and invalid lookup paths.",
  "preserved_prior_structure": "Method signatures, surrounding branch structure, and other production logic remain unchanged.",
  "possible_risks": "The bug is most visible through method-parameter tests that expect lookup results from non-empty maps."
}

```

Bug-Preserving Transformation Prompt

TASK:

Apply one bug-preserving transformation step after successful bug injection.

The transformation must keep the repository compilable and preserve the replay behavior that exposes the injected bug.

TRANSFORM SPEC 1:

- id: t_rare_profile_gate_v1
- paper_facing_operator: Rare Input Path Conditioning
- family: Control Flow
- name: Gate misroute behind rare profile
- description: restrict the misroute to a deterministic but low-probability input profile
- applicability: at_least_one_mid_level_helper_candidate
- forbidden_edits: changes_to_public_API_signatures, obvious_dead_branches_such_as_if_false
- must_preserve: compilation_success, public_api_signatures, replay_command_triggerability
- post_conditions: rare deterministic gate on the bug-related path

TRANSFORM SPEC 2:

- id: t_call_stack_deepen_v1
- paper_facing_operator: Helper Call Insertion
- family: Call Stack
- name: Deepen call stack with a natural helper extraction
- description: introduce a small helper method or lightweight wrapper to deepen the call stack


```

- applicability: at_least_one_mid_level_helper_candidate
- forbidden_edits: changes_to_public_API_signatures, broad refactors, formatting sweeps
- must_preserve: compilation_success, public_api_signatures, behavior_for_non_trigger_inputs
- post_conditions: additional_helper_in_call_chain

TRANSFORM SPEC 3:
- id: t_private_adapter_layer_v1
- paper_facing_operator: Call-Site Rerouting / Private Adapter Layer
- family: Call Stack
- name: Add a private adapter layer and reroute one call site
- description: introduce a private adapter method and reroute at least one internal call site through
  it
- applicability: at_least_one_internal_call_site_can_be_rerouted
- forbidden_edits: changes_to_public_API_signatures, obvious_dead_branches_such_as_if_false
- must_preserve: compilation_success, public_api_signatures, behavior_for_non_trigger_inputs
- post_conditions: additional_adapter_in_call_chain

TRACE CONTEXT SUMMARY:
{
  "trace_id": "org.apache.dubbo.common.url.URLParamTest",
  "test_name": "org.apache.dubbo.common.url.URLParamTest",
  "methods_count": 91
}

CANDIDATE ANCHORS:
- URLParam.hasMethodParameter
- URLParam.getMethodParameter
- URLParam.getMethodParameterStrict
- URLParam.initMethodParameters
- URLParam.size
- URLParam.isEmpty

EDIT BOUNDARIES:
- Do not modify tests unless explicitly allowed by the transform specification.
- Do not change public or protected API signatures.
- Avoid broad refactors, renames, or formatting sweeps.
- Keep diffs localized to trace-relevant production code.
- Preserve replay-command triggerability.

REQUIRED FINAL OUTPUT:
Append exactly one final line with this prefix:
TRANSFORM_REPORT_JSON: { ... }

Required keys:
ok, transform_id, changed_files, edit_sites, what_was_added_or_wrapped, what_was_preserved,
  what_was_replaced_or_overwritten, compatibility_with_previous_steps, expected_effect_on_stealth,
  expected_effect_on_trigger_depth

```

Bug-Preserving Transformation Report

```

TRANSFORM_REPORT_JSON:
{
  "ok": true,
  "transform_id": "t_rare_profile_gate_v1 + t_call_stack_deepen_v1 + t_private_adapter_layer_v1",
  "changed_files": [
    "dubbo-common/src/main/java/org/apache/dubbo/common/url/component/URLParam.java"
  ],
  "edit_sites": [
    "URLParam.getMethodParameter",
    "URLParam.hasMethodParameter",
    "URLParam.keyIndexToOffset",
    "URLParam.computeValueOffset",
    "URLParam.resolveParameterWithEncoding"
  ],
  "what_was_added_or_wrapped": "Added private helper and adapter methods around parameter lookup and
    offset resolution.",
  "what_was_preserved": "Public APIs, compilation, and replay-command triggerability.",
  "what_was_replaced_or_overwritten": "Direct parameter and offset resolution call sites were rerouted
    through private helpers.",
  "compatibility_with_previous_steps": "The transformation chain preserves the injected guard-
    inversion behavior.",
  "expected_effect_on_stealth": "The resulting structure resembles localized helper extraction and
    profile-aware lookup logic.",
  "expected_effect_on_trigger_depth": "The execution path now includes additional helper and adapter
    calls before the wrong value is observed."
}

```

PoC-Generation Prompt

SYSTEM:

You are an autonomous code agent operating inside a Java repository.

You are running a PoC-generation task for an already-injected production bug.

Your job is not to fix code.

Your job is not to improve the repository.

Your job is not to write a generally reasonable test.

Your only goal is to create a new Java testcase that reproduces the already-injected bug under the provided Maven verification command.

EVALUATION ATTEMPT BUDGET:

For each benchmark case, the evaluation protocol allows at most three PoC-generation attempts.

Each attempt must obey the same test-only constraints and use the provided replay command as the validation target.

HARD CONSTRAINTS:

1. Never modify production code under `src/main/java` or any non-test source root.
2. Never modify build files, `pom.xml` files, Gradle files, CI files, config files, or dependency files.
3. Only create or edit Java test files under `src/test/java`.
4. Prefer rewriting the official testcase file at the same path and with the same package and class name when that information is provided.
5. The testcase must materialize in the repository as a real `.java` file before verification.
6. The testcase must be designed to trigger the injected bug, not to validate normal behavior.
7. Do not output file blocks for an external parser. Directly write files into the repository workspace.
8. Keep the testcase minimal but bug-targeted.
9. Do not delete or modify unrelated files.

TASK PAYLOAD:

Generate a new Java testcase that triggers the already-injected bug.

Success criteria:

- A real Java testcase file exists in the repository under `src/test/java`.
- The testcase uses the correct package declaration.
- The testcase compiles.
- The provided verification command fails because the injected bug is triggered.
- The failure is a semantic test failure, not a compile error and not a harness failure.
- No production file or build configuration file is modified.

Repository: dubbo

Context setting: WithEC

Implementation group: B

Injected case run_id: inject__20260315T064448Z

Type ID: logic_predicate_inversion_in_guard

Bug type: Guard Predicate Inversion

Applied transforms: `t_rare_profile_gate_v1`, `t_call_stack_deepen_v1`, `t_private_adapter_layer_v1`

Bug-related production files:

- `dubbo-common/src/main/java/org/apache/dubbo/common/url/component/URLParam.java`

Official test path: `dubbo-common/src/test/java/org/apache/dubbo/common/url/URLParamTest.java`

Official test class: `org.apache.dubbo.common.url.URLParamTest`

Verification command: `./mvnw -q -pl dubbo-common -Dtest=org.apache.dubbo.common.url.URLParamTest test`

Injection-stage verify rc: 1

Type summary:

A boolean guard condition in production code was inverted, causing guarded URL-parameter lookup logic to execute under the wrong condition and leading to semantically incorrect behavior without changing public APIs.

Execution context block:

The target test reaches `URLParam` construction and method-specific parameter lookup.

Trace-relevant production methods include `URLParam.hasMethodParameter`, `URLParam.getMethodParameter`, `URLParam.getMethodParameterStrict`, `URLParam.initMethodParameters`, `URLParam.size`, and `URLParam.isEmpty`.

This execution context block identifies the path used during benchmark construction; it does not include the reference PoC implementation or hidden validation oracle.

What you must do:

- Read the bug context carefully.
- Inspect the relevant production and test files.
- Rewrite or create the testcase directly at the official test path.
- Keep the package name and class name correct.
- Exercise the suspected buggy path through `URLParam` method-specific parameter lookup.
- Write an oracle that fails under the injected bug.
- Run the provided verification command if needed.
- Leave no production-code or build-configuration changes.

Required final output:
At the very end, print exactly one single-line JSON object prefixed by:
TRIGGER_REPORT_JSON:

The JSON object must have this schema:

```
{
  "summary": "...",
  "suspected_bug_mechanism": "...",
  "inspected_production_files": [...],
  "inspected_test_files": [...],
  "selected_test_path": "...",
  "selected_test_class": "...",
  "trigger_strategy": [...],
  "expected_failure_signal": "...",
  "materialized_test_files": [...],
  "notes": "..."
}
```

PoC-Generation Report

```
TRIGGER_REPORT_JSON:
{
  "summary": "Generate a URLParam test that exercises method-specific parameter lookup after the transformed guard inversion.",
  "suspected_bug_mechanism": "A guard inversion and transformed lookup path cause method-specific parameter retrieval to return an incorrect value or miss a valid method entry.",
  "inspected_production_files": [
    "dubbo-common/src/main/java/org/apache/dubbo/common/url/component/URLParam.java"
  ],
  "inspected_test_files": [
    "dubbo-common/src/test/java/org/apache/dubbo/common/url/URLParamTest.java"
  ],
  "selected_test_path": "dubbo-common/src/test/java/org/apache/dubbo/common/url/URLParamTest.java",
  "selected_test_class": "org.apache.dubbo.common.url.URLParamTest",
  "trigger_strategy": [
    "Construct URLParam with multiple parameters so that method-specific lookup and transformed helper paths are exercised.",
    "Call hasMethodParameter and getMethodParameter on methods that should be present.",
    "Assert the expected valid lookup result so that the injected guard inversion becomes observable."
  ],
  "expected_failure_signal": "Assertion failure: expected method-specific parameter to be present or equal to the expected value, but the transformed buggy path returns an incorrect result.",
  "materialized_test_files": [
    "dubbo-common/src/test/java/org/apache/dubbo/common/url/URLParamTest.java"
  ],
  "notes": "The test is written only under src/test/java and does not modify production code or build configuration."
}
```

H Reproducibility Instructions

We report how to verify released POCGYM cases and rerun the construction and evaluation pipeline. Paths in the anonymous release may differ from the placeholders below, but the required inputs and outputs are unchanged.

H.1 Environment Requirements

POCGYM requires a Unix-like shell, Python 3, a Java runtime compatible with the target projects, and Maven or Maven-wrapper support. Running agent-based construction or evaluation additionally requires the corresponding model backend credentials. Because hosted agents may change over time, reproducibility is centered on archived case metadata, deterministic replay validation, repository-diff audits, and final success decisions rather than bit-identical agent text.

H.2 Verifying a Released Case

Each released case contains `case.json`, `bug.patch`, `buggy_repo/`, and an optional `execution_context.json` used for the `WithEC` setting. To verify one case, inspect `case.json`, enter the buggy repository snapshot, and run the recorded replay command verbatim:

```
cd <case>/buggy_repo
<recorded Maven replay command>
```

The case is valid only if the command fails because of the injected bug, rather than a missing dependency, compilation error, infrastructure failure, or unrelated test failure.

H.3 Reconstructing Benchmark Cases

Given clean repository templates, construction can be reproduced by collecting execution contexts, building an injection manifest, and running the paper-aligned construction wrapper:

```
python3 pocgym-trace-prep/scripts/collect_maven_execution_context.py \
--repo-template <benchmark-root>/<repo> \
--repo-name <repo> \
--official-test-class <test-class> \
--official-test-path <test-path> \
--workspace-root <out-root>/workspaces/trace \
--output-dir <out-root>/trace_assets/<trace-id>

python3 pocgym-inject/scripts/build_inject_manifest_from_traces.py \
--traces <out-root>/trace_assets/<trace-id>/selected_traces.json \
--commands <out-root>/trace_assets/<trace-id>/selected_commands.json \
--output <out-root>/manifests/inject.jsonl \
--repo-template-root <benchmark-root> \
--type-ids <bug-type-id> \
--transform-ids <transform-id-1> ...

python3 pocgym-inject/scripts/run_paper_aligned_inject_batch.py \
--project-root <project-root> \
--manifest <out-root>/manifests/inject.jsonl \
--workspace-root <out-root>/workspaces/inject \
--out-root <out-root>/runs/inject \
--dataset-root <out-root>/dataset \
--backend-mode <backend> \
--model-id <model-id>
```

The wrapper restores a clean workspace before each attempt, enforces the construction attempt budget, validates replay triggerability after transformations, and rejects cases without a production-code diff or with test/build-file modifications.

A successful construction run can be converted to the released-case format with:

```
python3 pocgym-full-workflow/scripts/package_review_case.py \
--case-summary <out-root>/runs/inject/cases/<case-key>/case_summary.json \
--case-id <case-id> \
--output-root <out-root>/release/cases \
--repo-id <repo> \
--type-category <category>
```

H.4 Running PoC-Generation Evaluation

Evaluation starts from packaged cases. The runner prepares a clean workspace, overlays the buggy production-code state, removes or rewrites the official target test, invokes the agent, and reruns the recorded replay command. First convert released cases to an evaluation manifest:

```
python3 pocgym-trigger/scripts/release_to_trigger_manifest.py \
--release-root <out-root>/release \
--repo-template-root <benchmark-root> \
--output-jsonl <out-root>/manifests/trigger.jsonl
```

Then run the two context settings:

```
python3 pocgym-trigger/scripts/run_paper_aligned_trigger_batch.py \
--project-root <project-root> \
```

```
--manifest <out-root>/manifests/trigger.jsonl \  
--out-root <out-root>/runs/trigger_<agent> \  
--groups A,B \  
--backend-mode <backend> \  
--agent-runner-script <runner-script> \  
--model-id <model-id>
```

Group A corresponds to NoEC, and group B corresponds to WithEC. The only intended difference is whether the archived execution-context summary is included. Neither setting exposes the reference PoC, hidden validation oracle, construction-time solution, or permission to modify production code.

The runner records prompts, agent outputs, generated tests, replay logs, diff audits, failure classifications, and success decisions. A PoC attempt is counted as successful only if it materializes a Java test under the allowed test root, the replay command fails with a semantic test failure exposing the injected bug, and no production or build files are modified.